

Subroutines and Control Abstraction

Parameter Passing Modes

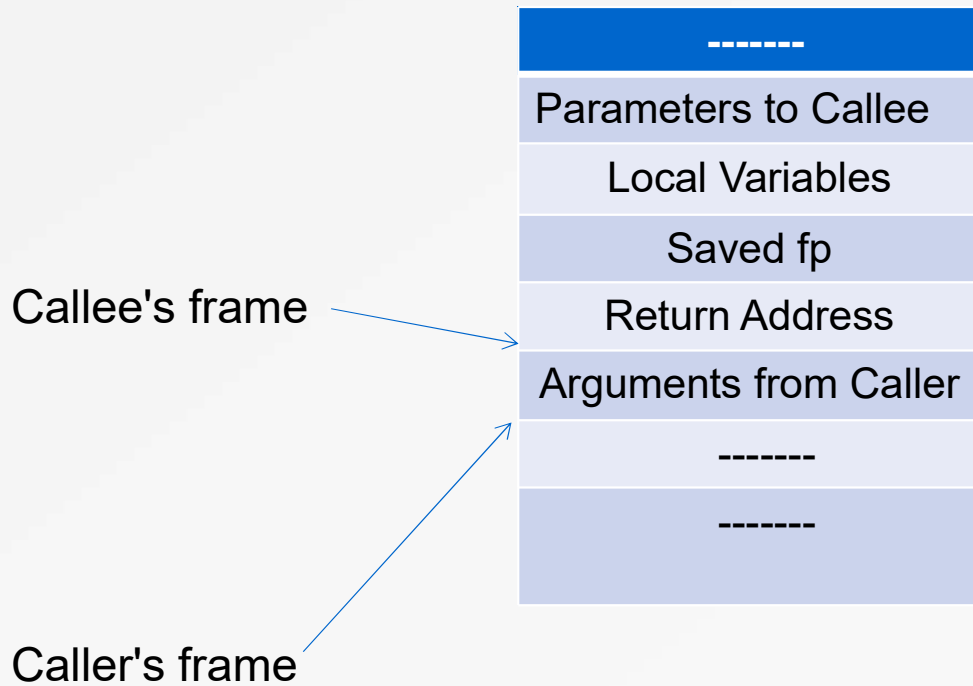
Calling Sequences

- The following will be executed during a subroutine call
 - Passing parameters
 - Saving return address
 - Changing program counter
 - Changing stack pointer to allocate space
 - Save registers
 - Changing frame pointer to point to a new frame
 - Executing initialization code for any new objects

Parameter Passing

- Formal parameters vs. Actual parameters
 - Parameter passing modes
 - Call by Value
 - Call by Reference (aliasing)
 - Call by Value/Result (copying)
 - Call by Sharing
 - Read-only Parameters
 - In, Out and In Out Parameters
 - Special Purpose Parameters
- 
- A decorative graphic on the right side of the slide, consisting of several overlapping, curved, wavy shapes in shades of light blue, yellow, and dark blue, creating a modern, abstract background element.

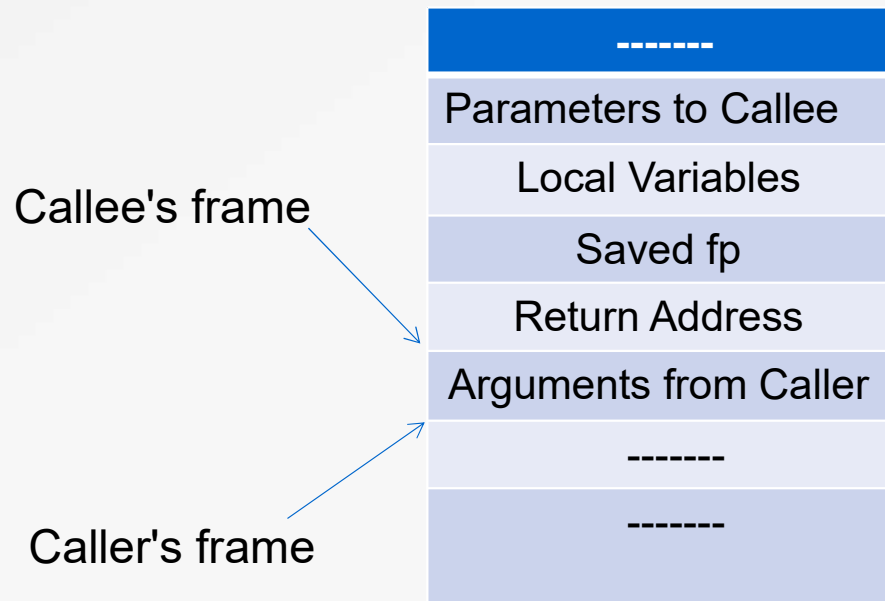
Pass by Value



- In Pass by Value, parameters are not in callee's frame, they are in caller's frame.
- Access the parameters by using fp and +ve offset.
- From the signature of functions, we know the space required for parameters.
- To avoid an excessive copy of the parameter, in C language, arrays are passed by reference (base address of an array is passed).
- Easy to simply pass an address to the called function rather than entire array.

Call-by-reference Cont..

- In Call by Reference, parameters are addresses and not the copy of parameters.
- C Language does not allow call by reference by default, whereas in C++, it is by default.
- In C language- programmer must explicitly pass arguments by reference for the called routine to make changes to the variables in caller's scope.
- Except arrays, they are passed by reference in C, base address of array is passed.



```
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }  
...  
swap(&v1, &v2);
```

Why references?

- To reflect the changes.
 - If the called routine is supposed to change the value of an actual parameter (argument), then the programmer must pass the parameter by reference.
 - If the called routine cannot modify the argument, the programmer can pass the parameter by value.
- To avoid excessive copying
 - Implementation of call-by-values requires copying actuals to formals, a potentially time-consuming operation when arguments are large.
 - Using references, we will require an extra level of indirection.

Call-by-sharing

- Call-by-value and call-by-reference for a language with a value model of variables (imperative languages).
- Everything is a reference in languages like Smalltalk, Lisp, ML, or Clu.
- In Java, it is call-by-value for variables of built-in type (all are values), and call-by-sharing for variables of user-defined class types (all are references).

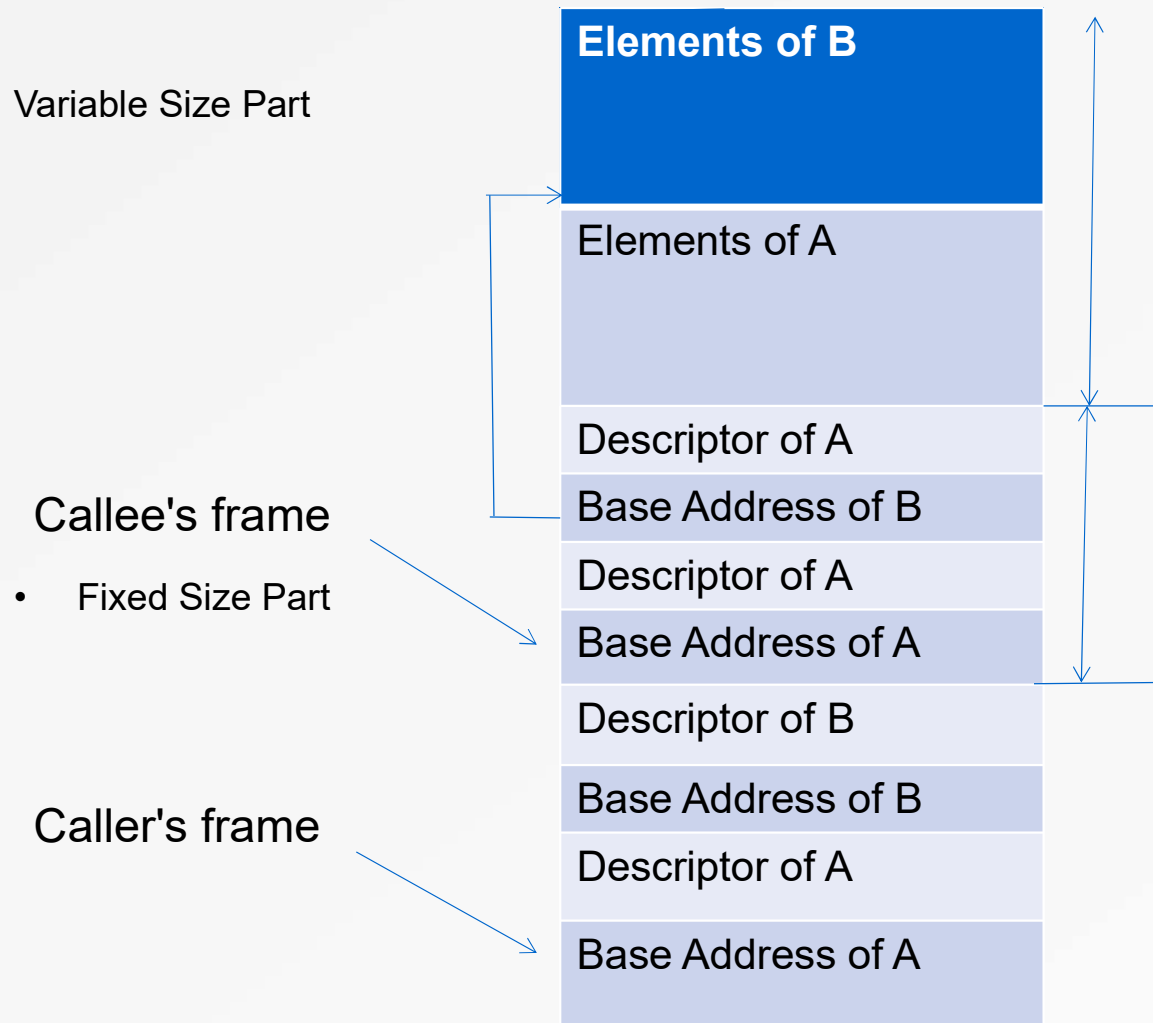
Special Purpose Parameters

- Conformant or Open Arrays
- Arrays
 - Static - lifetime is the entire program.
 - Stack allocation for arrays whose lifetime is an invocation of a subroutine.
 - Heap allocation for dynamically allocated arrays with more general lifetime.
- Storage management is more complex for arrays whose shape is not known until elaboration time, or whose shape may change during execution.
- The compiler must arrange not only to allocate space, but also to make shape information available at run time
- This information is needed for indexing of array elements.

Dope Vectors or Descriptors

- The dope vector will have
 - Number of dimensions
 - The lower bound and upper bound of every dimension
 - The size of each dimension other than the last
-
- The size of the last element is the size of the element type and that is known at compile time when the program is written.
- The dope vector is created by Compiler at compile time
- It is made available at runtime for accessing array elements

Frame structure for Conformant Arrays



- Every local object can be found using a known offset from the frame pointer.
- The stack frame is divided into a fixed size part and a variable-size part.
- An object whose size is statically known goes in the fixed-size part.
- An object whose size is not known until elaboration time goes in the variable-size part, and a pointer to it, together with a dope vector, goes in the fixed-size part.

Default Parameters

- Caller should not provide the arguments that are marked as default arguments. Ex. text_IO library package in Ada

```
type field is integer range 0..integer'last;
type number_base is integer range 2..16;
default_width : field      := integer'width;
default_base  : number_base := 10;
procedure put(item  : in integer;
              width : in field      := default_width;
              base  : in number_base := default_base);
```

- The maximum number of columns required to print an integer in decimal on the current machine

Variable Number of arguments

- In C, Lisp, Python, the user is allowed to define subroutines that take a variable number of arguments.
- Ex. the printf and scanf functions of C's stdio I/O library.
- In C, printf can be declared as follows:

```
int printf(char *format, ...)  
{ ...  
}
```

Variable Number of arguments

- `va_arg` function returns the value of the next omitted parameter.
- In `printf`, the `%X` placeholders in the format string are used to determine the type:
- `printf` can have a large switch statement, with one arm for each possible X.
- The arm for `%c` contains a call to `va_arg(args, char);`
- The arm for `%f` contains a call to `va_arg(args, double);`
- `va_end` does perform cleanup operations, for ex., deallocation of any heap space used for the `va_list`.